

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

C05: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Annexure A

Duration : 1 Hour
Total Marks:50

Design Task

Code of Conduct:

I Hemalakshmi H - 192571049 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Design Task

1. **Hybrid AI Recommendation Engine Design**
Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.

2. **Smart Disaster Detection System**
Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

3. **AI-Based Fraud Detection Framework**
Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.

4. **Autonomous Supply Chain Optimization System**
Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.

5. **Personalized Learning Analytics Platform**
Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed

Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined
Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

1. Hybrid AI recommendation engine design

A hybrid recommendation system combines **content-based filtering** (using item features like genre, tags, attributes) and **collaborative filtering** (using user–item interaction history such as ratings, clicks, purchases).

Using **PySpark RDDs**, user–item interactions are stored as RDDs of tuples $(u, s, eritemrating)$, and item profiles as $(i, temfeature_vector)$. Collaborative filtering (e.g., user–user or item–item similarity) is computed via RDD operations like map, join, and reduceByKey over distributed partitions, allowing similarity matrices and predicted ratings to be computed at scale. Content-based scores are computed by matching user preference vectors with item feature vectors using RDD transformations.

The **hybrid score** can be a weighted combination of collaborative and content-based scores, computed in parallel across the cluster.

Intelligent agents monitor user behavior (recent clicks, dwell time, skips, feedback) in real time, update user profiles, adjust weights between content-based and collaborative components, and re-rank recommendations dynamically. Agents can also explore new items (exploration–exploitation) and adapt to concept drift (changing tastes) by periodically retraining models or refreshing similarity metrics.

2. Smart disaster detection system

The system ingests **multi-source data streams**: satellite images, IoT sensor readings (water level, seismic activity), and social media posts. Each stream is processed as RDDs—e.g., satellite tiles as $(l, ocatationimage_features)$, sensors as $(s, e, nsor_idtimestampvalue)$, and social media as $(g, e, o_tagtext_featuressentiment)$.

Data fusion is done by joining and aggregating these RDDs on spatial and temporal keys (e.g., region + time window). Image features (from pre-trained CNNs), sensor anomalies, and social media spikes are combined to compute a **disaster risk score** per region using RDD transformations like groupByKey, reduceByKey, and windowed aggregations.

The **system architecture** includes:

- **Ingestion layer** (stream receivers, Kafka, etc.) feeding PySpark.
- **Processing layer** (RDD-based feature extraction and fusion).
- **Decision layer** with intelligent agents.

Intelligent agents coordinate by:

- Monitoring fused risk scores across regions.
- Triggering early warnings when thresholds are crossed.
- Communicating with response agents (evacuation planning, resource allocation).
- Learning from feedback (false alarms vs. true events) to refine thresholds and models over time.

3. AI-based fraud detection framework

Financial transactions are represented as RDDs of $(t, r, a, n, s, a, c, t i o n_i d, u s e r_i d, a m o u n t, l o c a t i o n, t i m e, m e r c h a n t, f e a t u r e s)$. **Anomaly detection algorithms** (e.g., clustering-based, statistical outlier detection, isolation-based methods) are implemented using distributed RDD operations.

For example, feature vectors are normalized and grouped by user or merchant using `keyBy` and `groupByKey`. Statistical profiles (mean, variance, typical transaction patterns) are computed via `reduceByKey`. Transactions deviating significantly from these profiles (e.g., z-score beyond threshold, unusual location/time combinations) are flagged as anomalies. Clustering algorithms (like k-means) can be implemented in PySpark, where each iteration uses RDD transformations to assign points to clusters and update centroids.

Intelligent agents collaborate by:

- Monitoring flagged transactions in real time.
- Combining multiple signals (rule-based checks + anomaly scores + historical fraud patterns).
- Triggering alerts to human analysts or automated blocking systems.

- Incorporating feedback (confirmed fraud vs. false positives) to update models, thresholds, and rules, thus continuously improving detection accuracy. Agents can specialize (per region, per merchant type) and share learned patterns across the network.

4. Autonomous supply chain optimization system

Supply chain data—inventory levels, orders, shipments, demand history, lead times—is stored as RDDs (e.g., $(w, a, r, ehousedproductstocktimestamp)$, $(r, o, utecosttime)$, $(p, roductdemand_history)$). **RDD transformations** like map, filter, join, and aggregateByKey support large-scale processing: forecasting demand per product, computing reorder points, and evaluating logistics costs across many routes and warehouses.

Demand forecasting models (e.g., time-series or regression) are trained using historical data in distributed fashion. Optimization tasks (e.g., choosing suppliers, routes, inventory policies) use aggregated metrics computed via RDDs.

Multiple **intelligent agents** interact in a decentralized way:

- **Inventory agents** decide reorder quantities based on forecast and safety stock.
- **Logistics agents** choose optimal routes and carriers considering cost and delivery time.
- **Demand agents** monitor market signals and update forecasts.

Agents share information (constraints, costs, forecasts) and negotiate decisions to minimize overall cost and improve efficiency. Because data and decisions are distributed, the system scales to many products and locations while adapting to changes (delays, demand spikes) in near real time.

5. Personalized learning analytics platform

Student interactions quiz scores, time spent on content, clickstream, forum activity—are stored as RDDs of $(s, t, u, d, ent_idactivity_typeresource_idtimestampperformance_metrics)$. PySpark RDDs enable **scalable analytics** across thousands of learners: computing engagement metrics, learning paths, and performance trends using groupByKey, reduceByKey, and windowed aggregations.

Models for **learning analytics** (e.g., predicting dropout risk, recommending next content, estimating mastery level) are trained on these RDDs. Adaptive learning is achieved by mapping each student to a **profile vector** (skills mastered, preferred content type, pace) and matching them to suitable resources.

Intelligent agents support personalization by:

- Monitoring each student's real-time behavior (struggles, rapid progress, inactivity).
- Adjusting difficulty, content type, and sequence dynamically.
- Providing immediate feedback (hints, remedial content, challenges).
- Coordinating across many students to balance resource usage (e.g., live sessions, tutor allocation).

Agents learn from outcomes (improved scores, reduced dropout, engagement changes) and refine recommendation policies, making the platform increasingly effective for large-scale, personalized learning.